



THE CHIPS
TO SYSTEMS
CONFERENCE

SPONSORED BY:



SiTA: Exploiting Sparsity in Tensor-Product for Accelerating Quantum Readout Error Mitigation

Hanyu Zhang¹, Zhiwei Ye², Liqiang Lu^{1*}, Kaiwen Zhou¹, Fangxu Guo¹,

Siwei Tan¹, Fangxin Liu³, Size Zheng⁴, Jianwei Yin¹

¹Zhejiang University ²China Mobile (Suzhou) Software Technology Co., Ltd

³Shanghai Jiao Tong University ⁴Tsinghua University



**THE CHIPS
TO SYSTEMS
CONFERENCE**

SPONSORED BY:



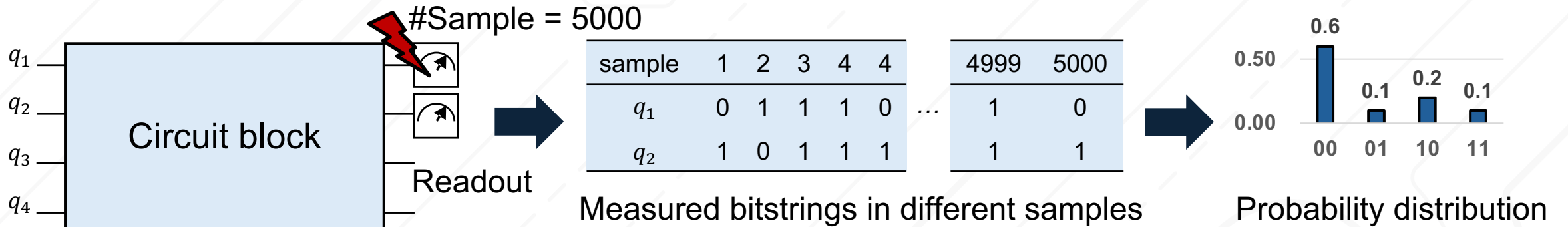
Outline of Presentation

- Background
- Motivation
- Sparse Dataflow for Readout Error Mitigation
- Architecture Design
- Evaluation



Quantum Readout

- Quantum readout **reads the information from quantum bits to classical bits.**



However, the quantum readout process suffers from readout error.



Readout Error

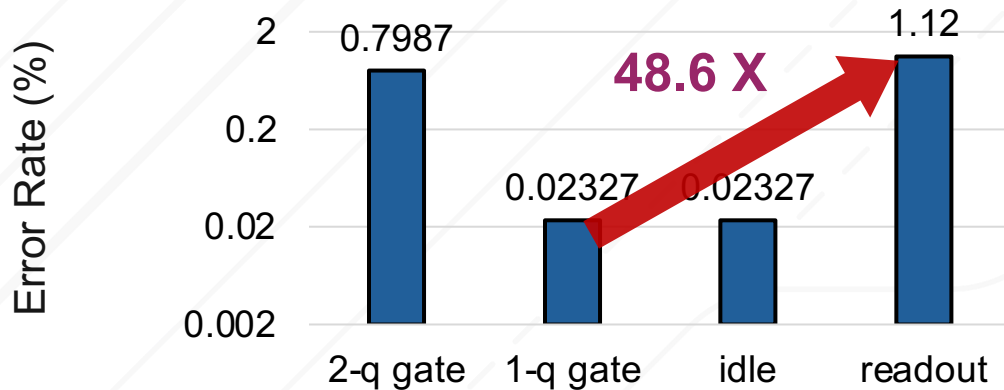
■ Error sources

Long readout
latency

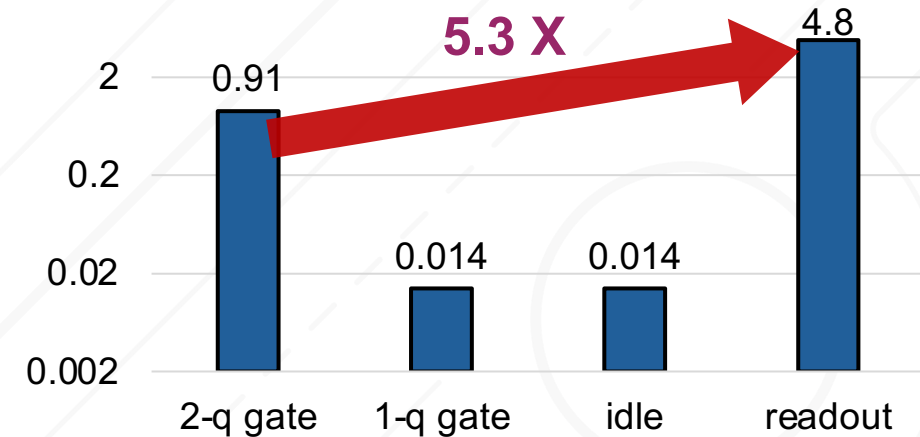
Crosstalk

Imperfect
discriminator

■ Readout error is **significant on current quantum hardware**.



Noise on 127-qubit IBM Sherbrooke quantum device



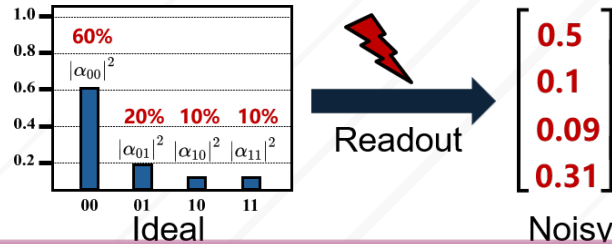
Noise on 10-qubit Tianmu quantum device

It is essential to mitigate readout errors to obtain reliable results.

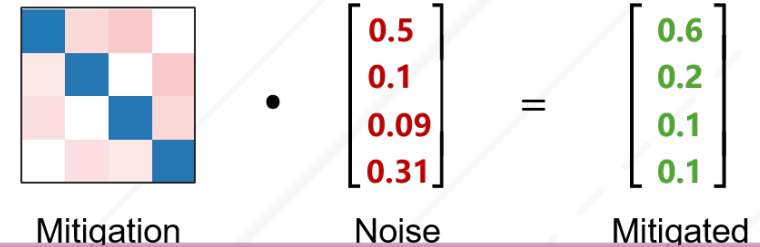


Matrix-Based Readout Mitigation

Readout with Noise ($P_{noisy} = N \cdot P_{ideal}$)

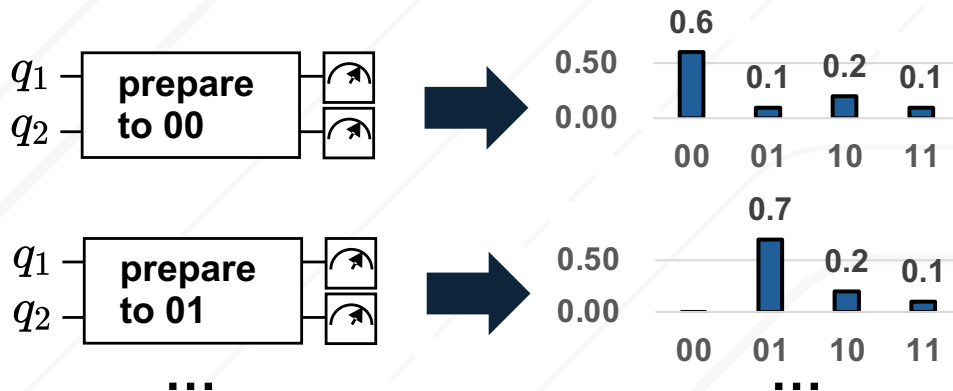


Perform MVM ($P_{miti} = M \cdot P_{noisy}$)



How to obtain the mitigation matrix?

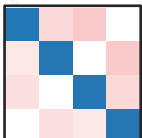
① Prepares qubits to different basis states and apply measurement.

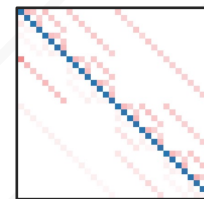


② Fill the noisy matrix

$$N = \begin{bmatrix} 0.6 & 0.1 & 0.2 & 0.1 \\ 0 & 0.7 & 0.2 & 0.1 \\ 0.2 & 0.2 & 0.6 & 0 \\ 0 & 0.1 & 0.1 & 0.8 \end{bmatrix}$$

③ Inverse the noisy matrix

$$M = N^{-1} = \text{Mitigation matrix}$$




5-qubit mitigation matrix: $2^5 \times 2^5$

10-qubit mitigation matrix: $2^{10} \times 2^{10}$

The size exponentially increases!

Matrix-based readout mitigation is limited in **spatial scalability** !



Tensor-Product-Based Mitigation

Key Idea: Tensor-Product Approximation

$$P_{miti} = M \cdot P_{noisy}$$

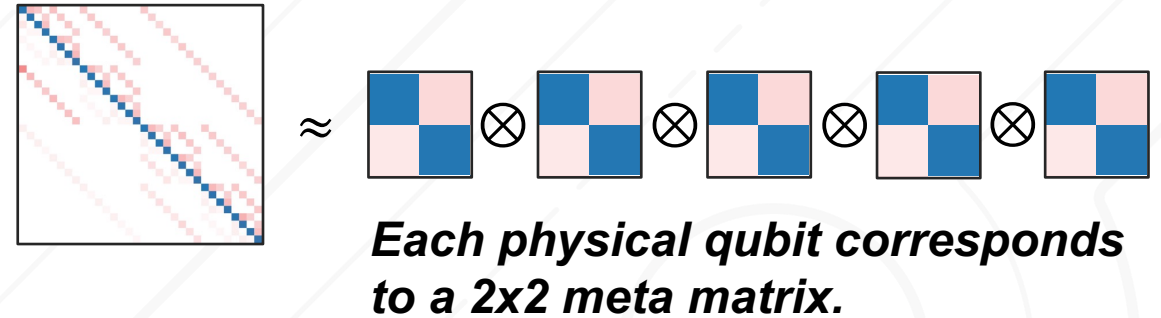


$$P_{miti} = (\underbrace{M_1 \otimes M_2 \otimes \dots \otimes M_k}_{\text{sub-mitigation matrices}}) \cdot P_{noisy}$$

Each submatrix shows exponential reduction in size.

Qubit Group and Sub-Mitigation Matrix

IBM Mthree ^[1] and Google IBU ^[2].



Good scalability, **low fidelity (crosstalk-unaware)**.

[1] Nation, Paul D., et al. "Scalable mitigation of measurement errors on quantum computers." PRX Quantum 2.4 (2021): 040326.

[2] Pokharel, Bibek, et al. "Scalable measurement error mitigation via iterative bayesian unfolding." Physical Review Research 6.1 (2024): 013187.



THE CHIPS
TO SYSTEMS
CONFERENCE

SPONSORED BY:

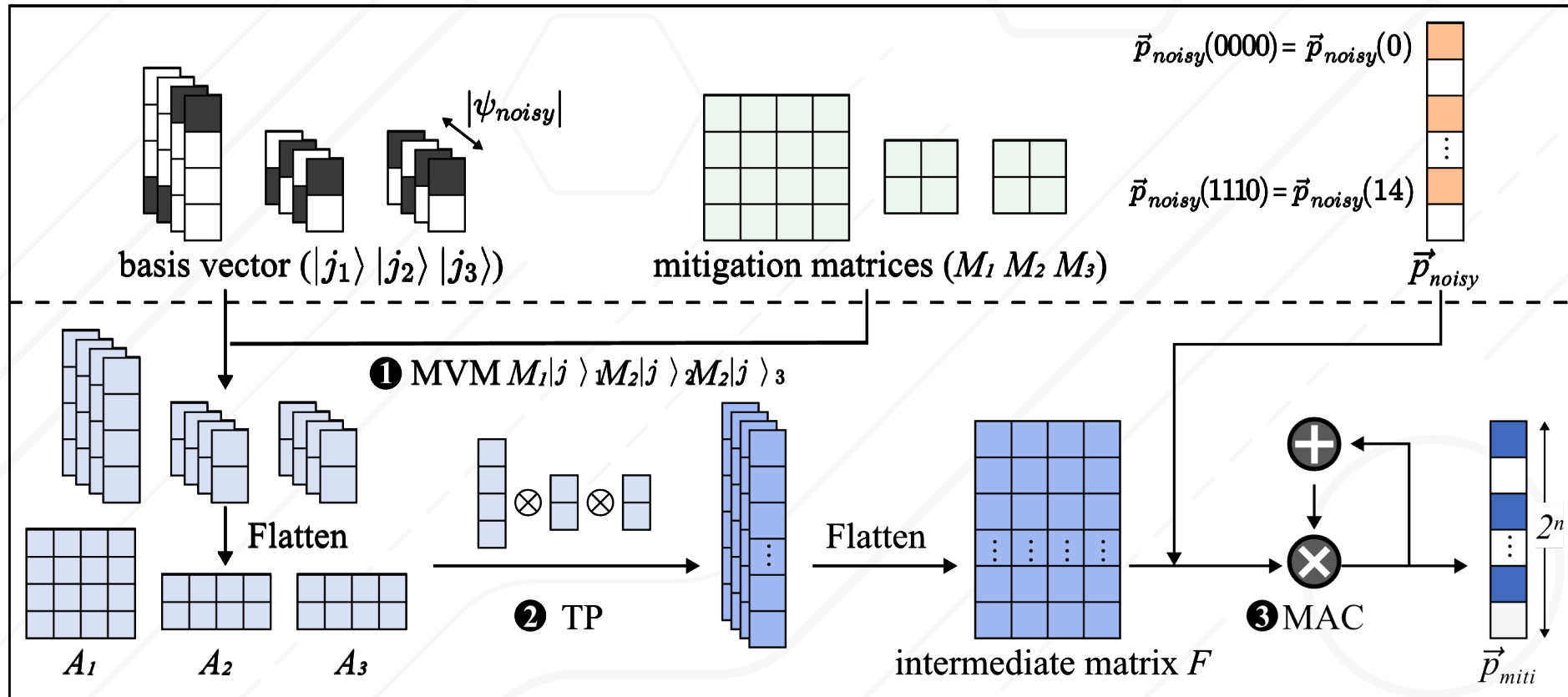


Outline of Presentation

- Background
- **Motivation**
- Sparse Dataflow for Readout Error Mitigation
- Architecture Design
- Evaluation



Computation Flow

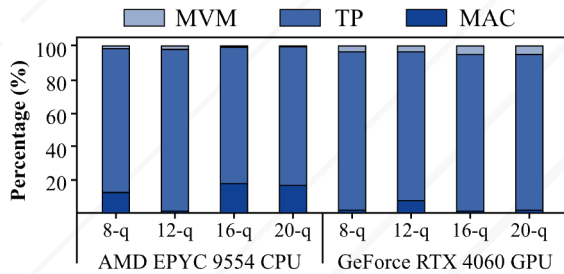


3 Multiply-and-Accumulate (MAC)



Sparse Opportunities

Latency Breakdown



AMD EPYC 9554 CPU:

- MVM, TP, and MAC account for 5.6%, **90.7%**, and 1.8%, respectively.

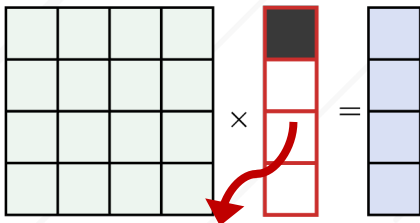
GeForce RTX 4090 GPU:

- MVM, TP, and MAC account for 5.6%, **92.6%**, and 1.8%, respectively.

The TP operator is the performance bottleneck.

Input Sparsity

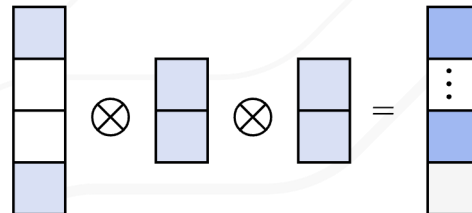
MVM



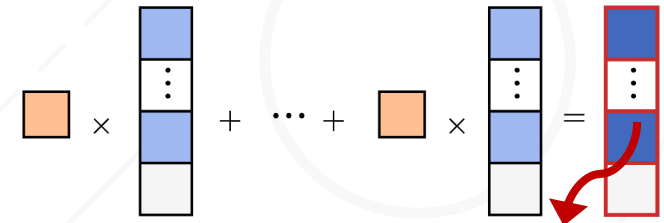
Basis states are unit vectors.

Output Sparsity

TP



MAC



Only a few basis states have nonzero amplitudes.

How can sparsity be exploited to accelerate computation?



THE CHIPS
TO SYSTEMS
CONFERENCE

SPONSORED BY:



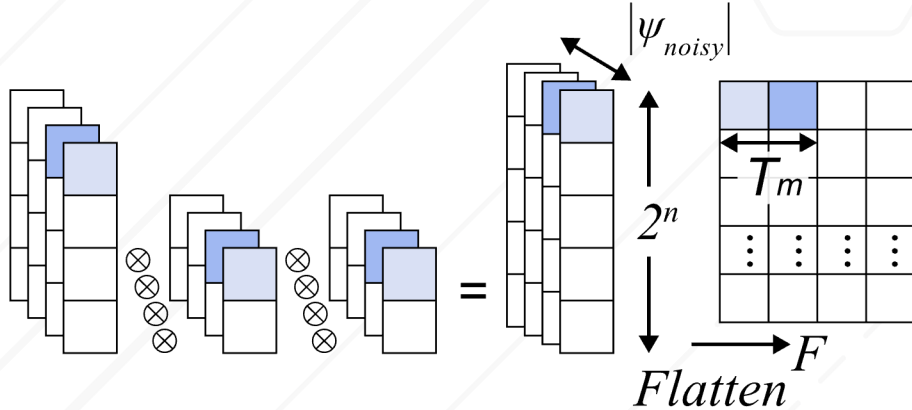
Outline of Presentation

- Background
- Motivation
- **Sparse Dataflow for Readout Error Mitigation**
- Architecture Design
- Evaluation



Parallelization for Tensor-Product

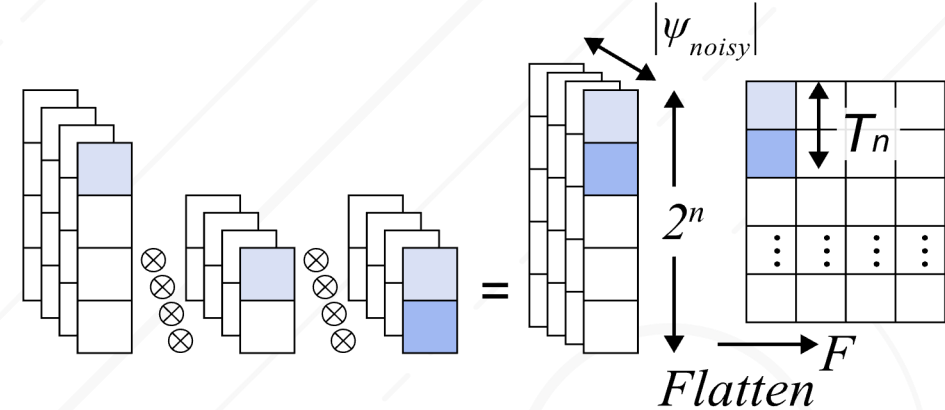
Probability-Level Parallelism



$$A_1(0,0) \times A_2(0,0) \times A_3(0,0) = F(0,0)$$

$$A_1(0,1) \times A_2(0,1) \times A_3(0,1) = F(0,1)$$

State-Level Parallelism



$$A_1(0,0) \times A_2(0,0) \times A_3(0,0) = F(0,0)$$

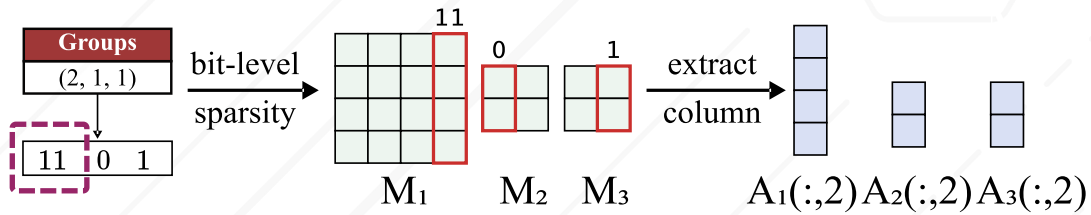
$$A_1(0,0) \times A_2(0,0) \times A_3(1,0) = F(1,0)$$

By combining the two parallelization strategies, the system **accelerates TP** computation at one tile per cycle.



Leveraging Sparsity

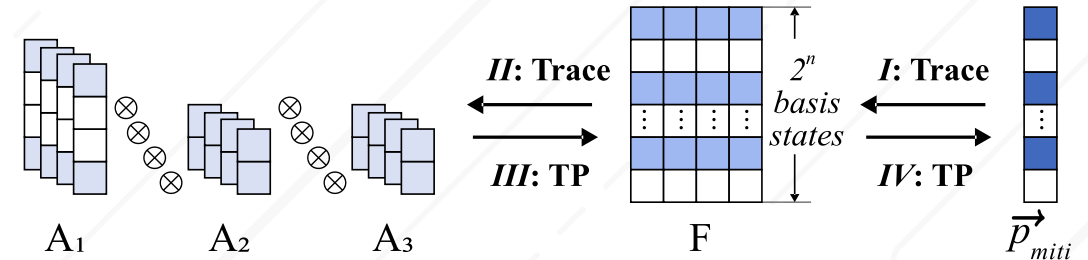
Leveraging Input Sparsity



$[0, 0, 0, 1]$ is a unit vector

- Unit vectors (**Basis states**) are inherently sparse.
- Transform matrix-vector multiplication (MVM) into **column extraction**.

Leveraging Output Sparsity



- Two-level backtracking is used to identify the elements with non-zero values.
- Leverage output sparsity to compute only **nonzero values**.



THE CHIPS
TO SYSTEMS
CONFERENCE

SPONSORED BY:



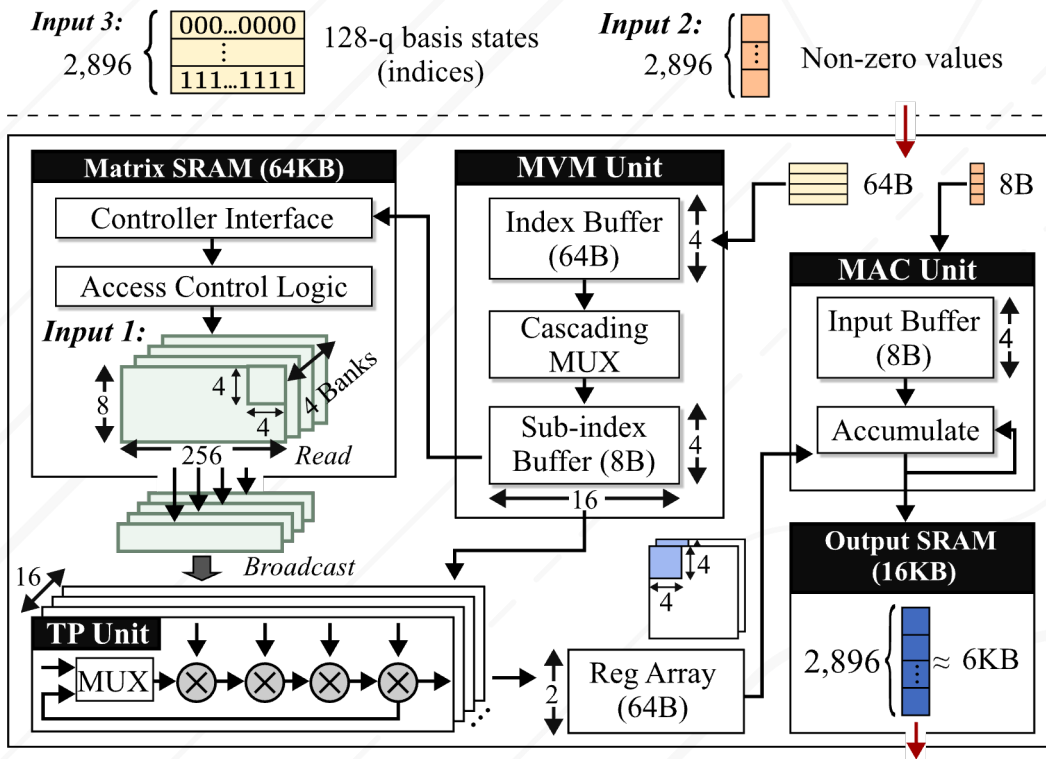
Outline of Presentation

- Background
- Motivation
- Sparse Dataflow for Readout Error Mitigation
- **Architecture Design**
- Evaluation



Overall Architecture

Overview of SiTA accelerator



Memory Hierarchy:

- **On-chip memory:** mitigation matrix
- **Off-chip memory:** basis states and non-zero values

Computing Modules:

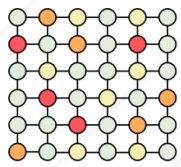
- MVM Unit, TP Unit and MAC Unit

**Tailored memory hierarchy +
Specialized computing modules!**

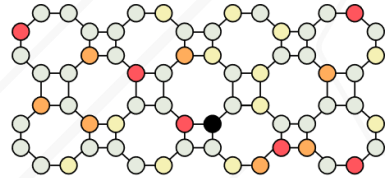


Dynamic Multiplier Chain

- A flexible processing unit is needed for chained multiplications of varying lengths:

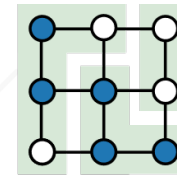


36 qubits

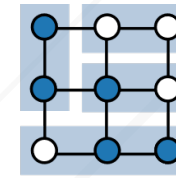


79 qubits

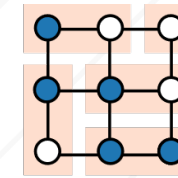
Systems with different numbers of bits



3 groups



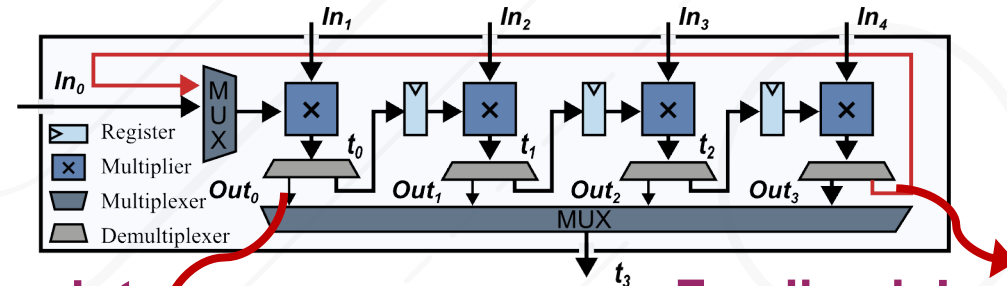
4 groups



5 groups

Systems with different bit grouping schemes

Dynamic Multiplier Chain:



Selectable output point

Feedback Loop

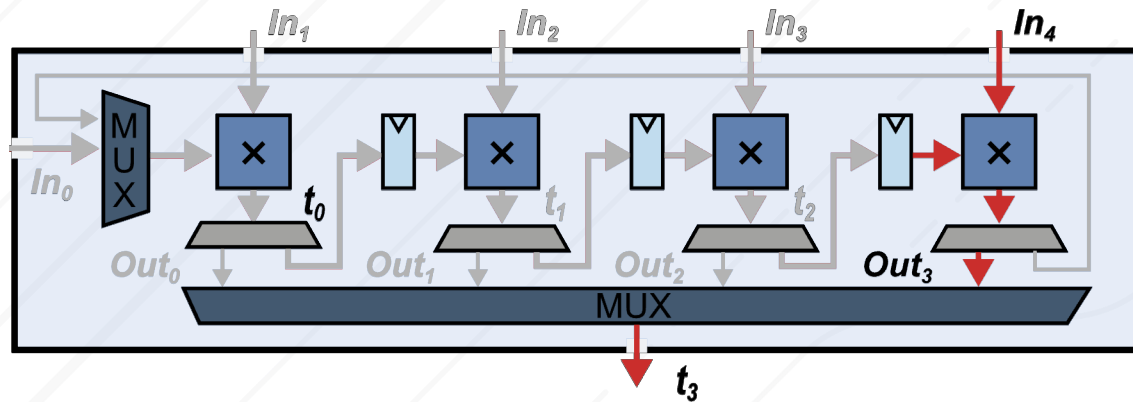
- ❑ **Demultiplexer:** outputs the current result or forwards it to the next multiplier.
- ❑ **Multiplexer:** feeds back the intermediate result for successive multiplications.



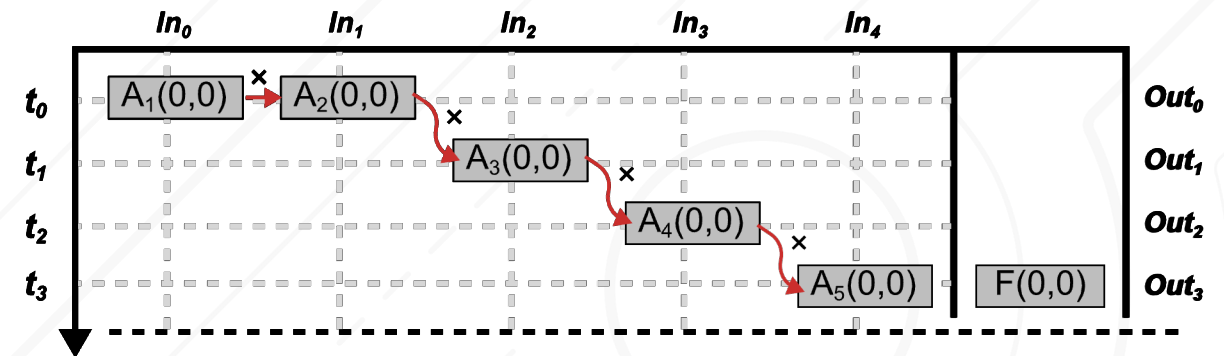
Dynamic Multiplier Chain

Example: An 8-qubit system is divided into 5 qubit groups:

$$F(0,0) = A_1(0,0) \times A_2(0,0) \times A_3(0,0) \times A_4(0,0) \times A_5(0,0)$$



(a) Hardware Architecture



(b) Processing Pipeline



THE CHIPS
TO SYSTEMS
CONFERENCE

SPONSORED BY:



Outline of Presentation

- Background
- Motivation
- Sparse Dataflow for Readout Error Mitigation
- Architecture Design
- Evaluation



Fidelity

- **Benchmarks:** BV, GHZ, FALQON, VQE, and QAOA
- **Baselines:** IBM Mthree ^[1], Google IBU ^[2], QuFEM ^[3], SpREM ^[4]
- **Quantum hardware platform:** 79-qubit Rigetti quantum platform (average readout/CZ/XY gate errors: 5.5%/5.2%/4.4%)

Workload	Raw	Mthree	SpREM	IBU	QuFEM	SiTA
BV-16	0.22	0.47	0.48	0.60	0.61	0.68
GHZ-16	0.20	0.34	0.35	0.35	0.45	0.49
QAOA-10-2L	0.19	0.20	0.20	0.21	0.25	0.29
QAOA-12-2L	0.12	0.13	0.13	0.15	0.19	0.24
VQE-10-2L	0.39	0.40	0.41	0.42	0.48	0.51
VQE-12-2L	0.28	0.28	0.29	0.31	0.34	0.35
FAL.-10-35L	0.37	0.38	0.38	0.39	0.40	0.42
FAL.-12-30L	0.23	0.25	0.26	0.28	0.31	0.32

□ SiTA achieves an average fidelity improvement of **1.1× to 1.4×**.

Mitigating only nonzero probabilities avoids noise amplification and improves fidelity.

[1] Nation, Paul D., et al. "Scalable mitigation of measurement errors on quantum computers." PRX Quantum 2.4 (2021): 040326.

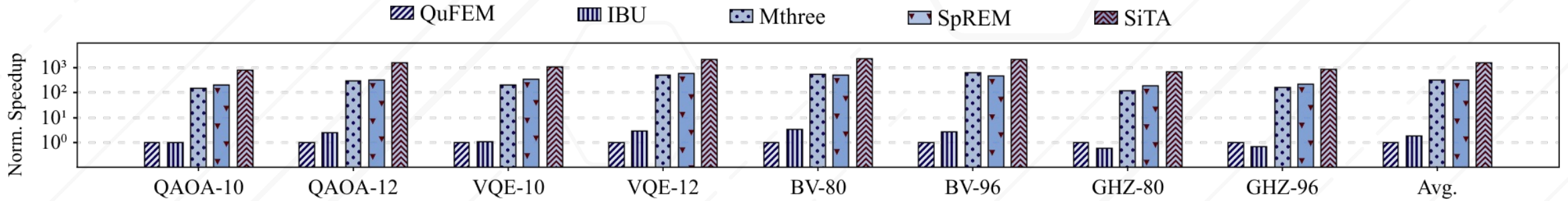
[2] Pokharel, Bibek, et al. "Scalable measurement error mitigation via iterative bayesian unfolding." Physical Review Research 6.1 (2024): 013187.

[3] Tan S, Lu L, Zhang H, et al. Qufem: Fast and accurate quantum readout calibration using the finite element method[C]//ASPLOS. 2024: 948-963.

[4] Zhang H, Lu L, Tan S, et al. Sprem: Exploiting hamming sparsity for fast quantum readout error mitigation[C]//DAC. 2024: 1-6.

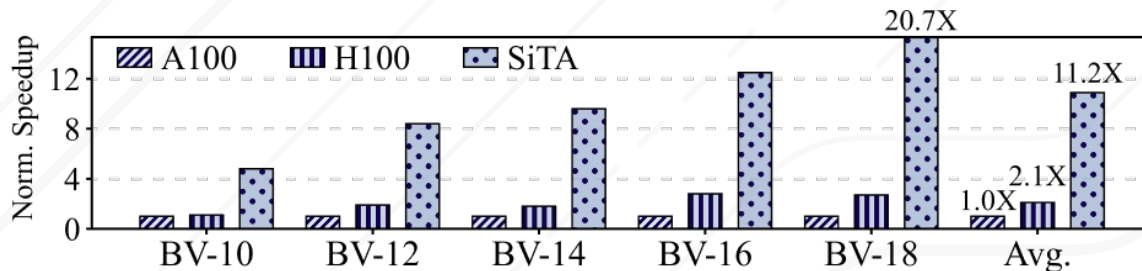


Speedup



Comparison with SOTA mitigation methods

- SiTA achieves an average speedup of **5.0×**, **4.9×**, **844×**, and **1604×** over SpREM, Mthree, IBU, and QuFEM, respectively.



Comparison with the cuSPARSE library

- 11.2×** speedup over the A100 GPU
- 5.3×** speedup over the H100 GPU

Sparse acceleration + customized accelerator!